



University
of Basel

Chapter 5.3: Parallel Programming – Distributed Memory

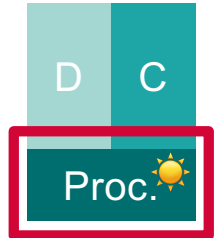
Foundations of Distributed Systems

Fall Semester 2025

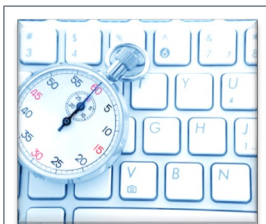
Prof. Dr. Florina Ciorba



Addressed in The **Processing** Lecture Series



• Goals



Performance

```
00111101001010101
01001010100010101
01010ERROR011010
10100010100110001
01100010100100111
10100010100110001
```

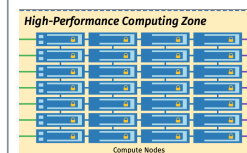
Resiliency



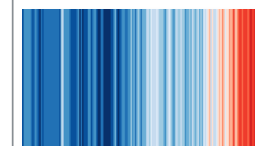
**Extreme
scalability**



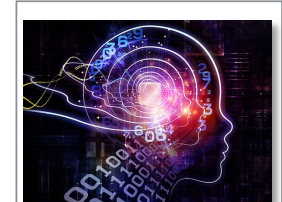
Energy



Security



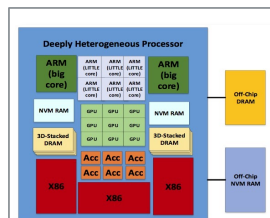
Sustainability



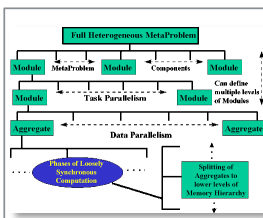
**Convergence
HPC+Data+AI**

• Challenges

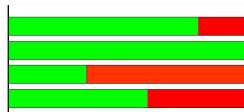
Chapter 5.3: Parallel programming



Heterogeneity



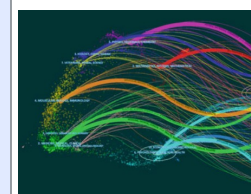
**Multilevel
parallelism**



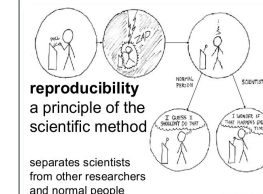
Load imbalance



Programming



Visualization



Reproducibility

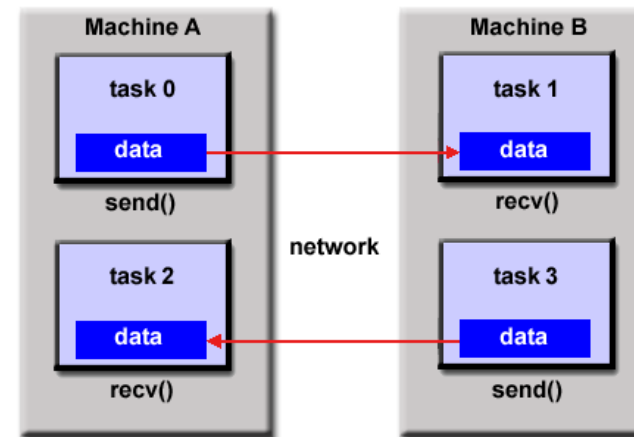
Overview of Chapter 5.3: Parallel Programming

Introduction to Parallel Programming

a Shared-Memory Programming: OpenMP Introduction

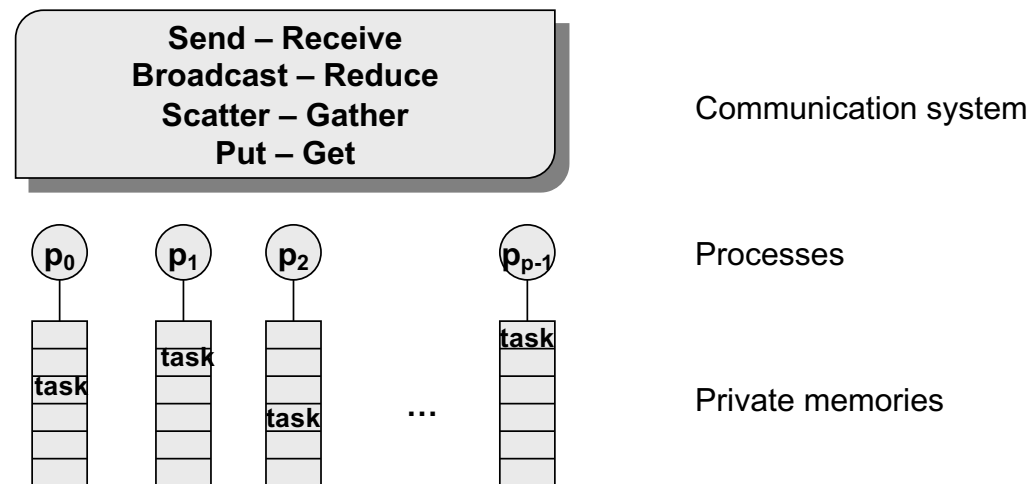
b Distributed-Memory Programming: MPI Introduction

Parallel programming model 4:
Distributed-Memory | Message passing



Message Passing Programming: Principles (1/2)

- $p > 1$ processes, each with its own **exclusive address space** (NOMA)
- Data and work are **explicitly** partitioned between p_i and placed in their **private memory**
- All interactions (read-only or read/write) require
 - **Explicit cooperation** of 2 (or more) processes: **sender** and **receiver** → two-sided communication
 - *Implicit cooperation* of 2 processes: source and target → one-sided communication (more in Spring HPC)
 - Through the communication system (involves communication library and underlying network interconnect)



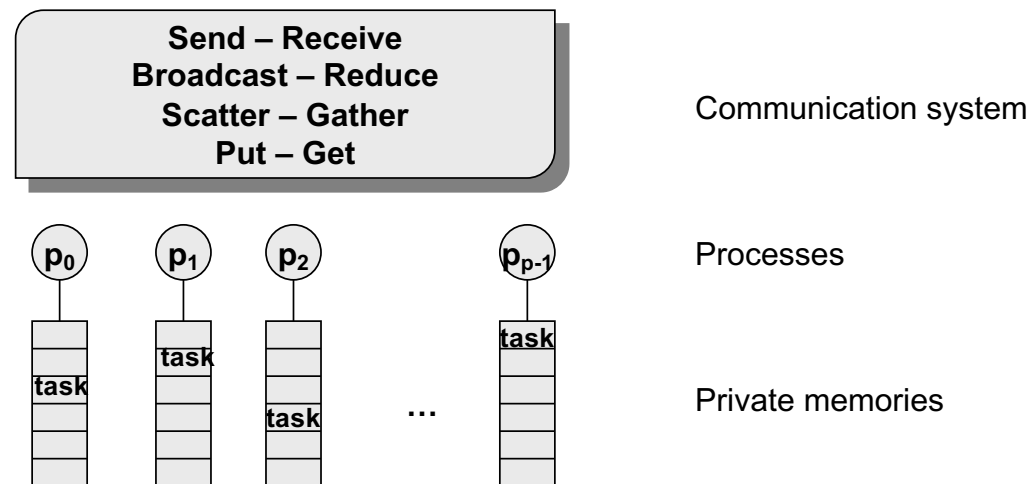
Message Passing Programming: Principles (2/2)

Increased programmer responsibilities especially **all** program organization:

- Organization and distribution of data+work into tasks (Step 1), preserving their interactions (Step 2) and organizing their mapping to processes (Step 3)

Message Passing supports and realizes inter-process synchronization & communication

Needs a certain development time but **delivers greater performance** than shared-memory parallel programming or sequential programming.



Message Passing Programming

A Note on **Loosely Synchronous** vs. Asynchronous Execution

SIMD (different paradigm, i.e., vectorization, e.g., on vector units of CPUs, on GPUs)
=> **Synchronous** execution

Most Message Passing programs are written using SPMD (*single program multiple data*) model
=> Recall programming model 6?

Loosely synchronous execution paradigm

- Tasks or subsets of tasks **synchronize** to perform interactions
- In-between these interactions, tasks execute completely *asynchronously*

(This is called *bulk synchronous parallelism* or **BSP**)

 => **Parallel computing** is **loosely synchronous** => Our focus in FDS-HPC & chapter 5

Asynchronous execution paradigm

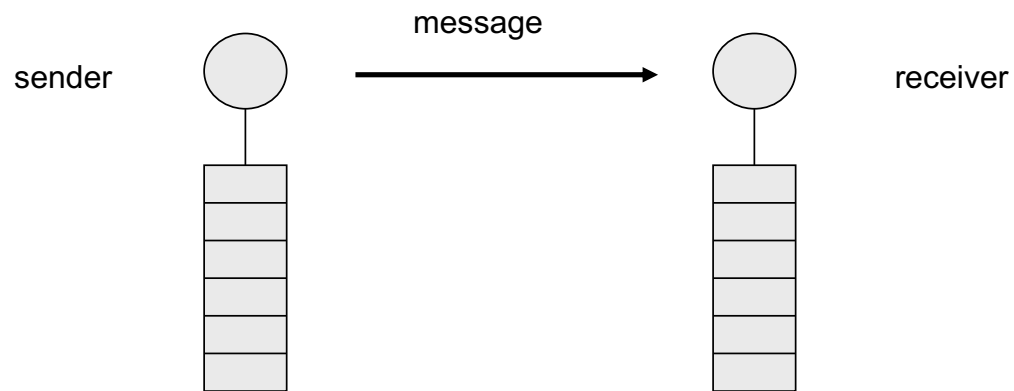
- All concurrent tasks execute pretty much asynchronously, and encountered in

=> Embarrassingly **parallel computing**

=> (Fully) **Distributed computing**

Basic Message Passing Operations

Send and Receive



send(void *sendbuf, int nelems, int dest)

receive(void *recvbuf, int nelems, int source)

Basic Message Passing Operations

Send and Receive: Semantics Behind

P0

```
a = 100;  
send(&a, 1, 1);  
a = 0;
```

P1

```
receive(&a, 1, 0)  
printf("%d\n", a);
```

What should P1 receive ?

Common sense dictates that 100 should be received.

But everything depends on **semantic aspects**:

Does **send()** **block** P0 until data delivered to P1 or does P0 **continue** execution?

Different implementation protocols of **send()** will yield different results.

Message Passing Operations

Space of Communication Protocols

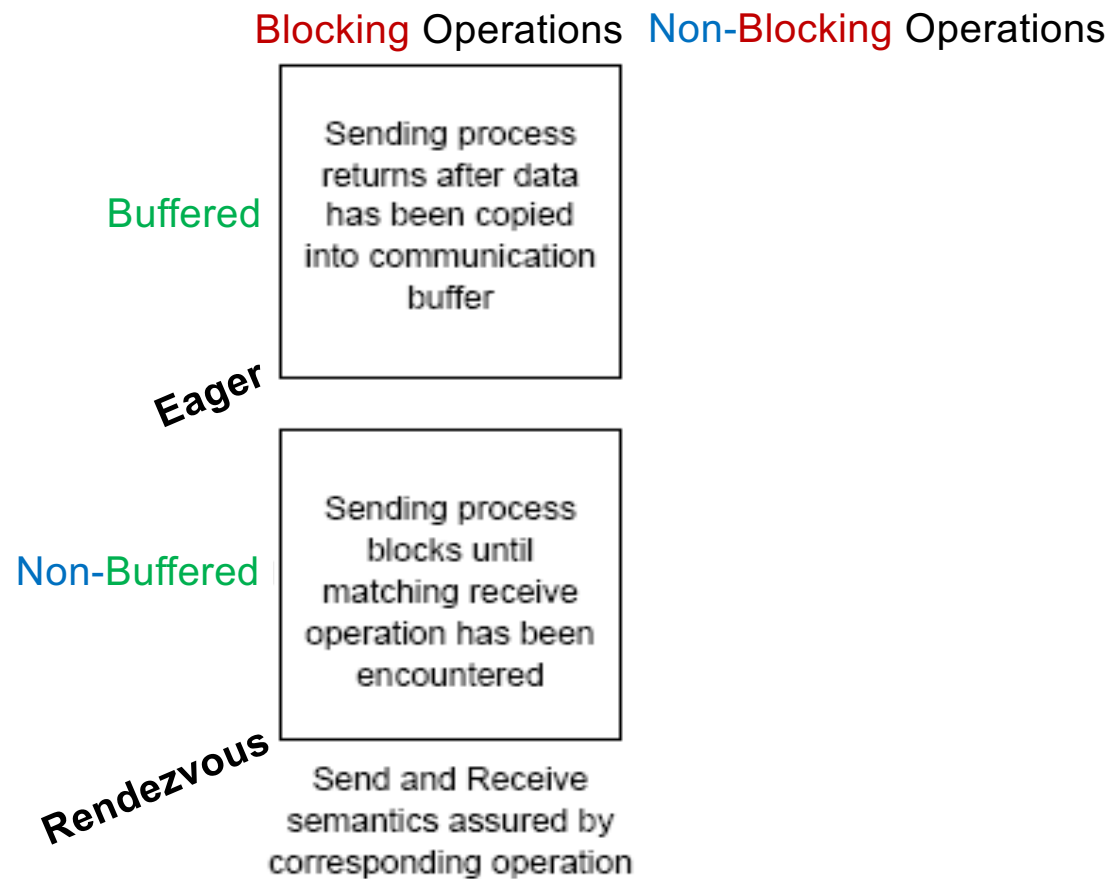
Blocking Operations Non-Blocking Operations

Buffered

Non-Buffered

Message Passing Operations

Space of Communication Protocols



Message Passing Operations

Space of Communication Protocols

Blocking Operations Non-Blocking Operations

Buffered

Sending process
returns after initiating
DMA transfer to
buffer. This operation
may not be
completed on return

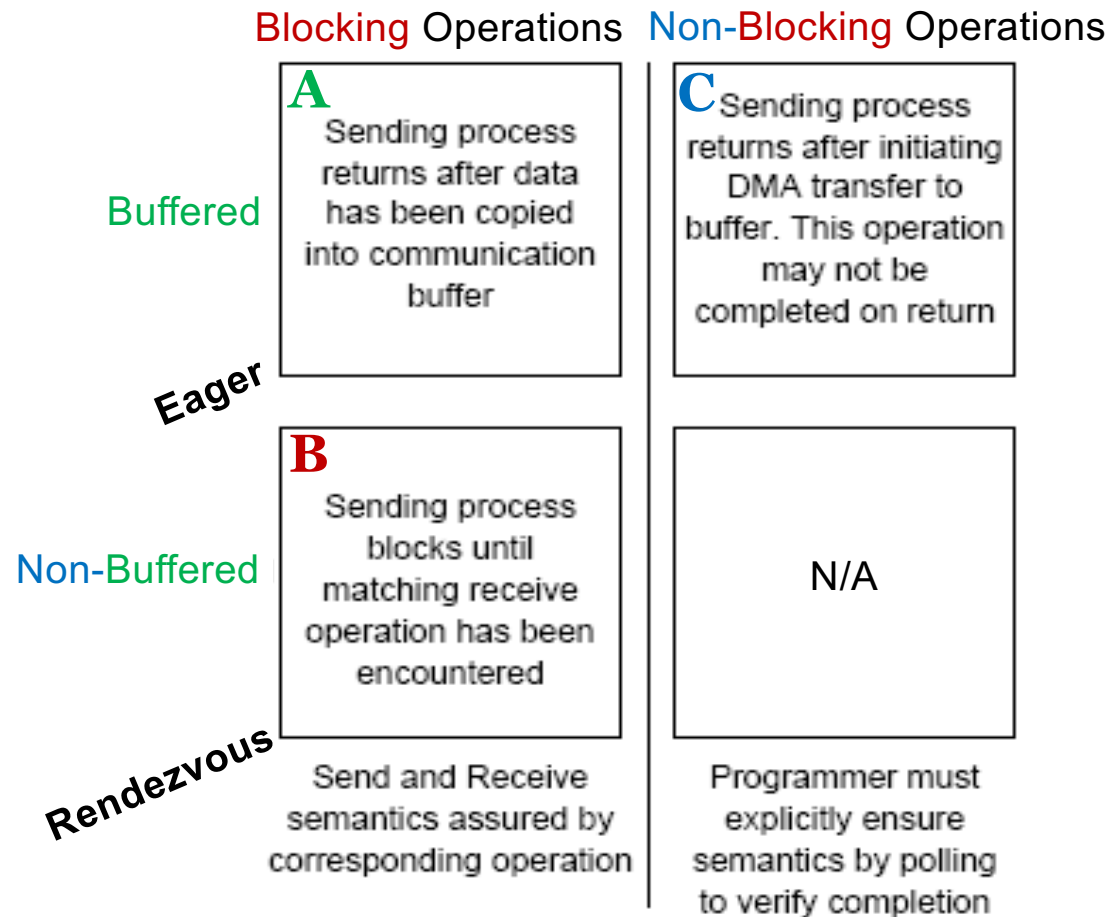
Non-Buffered

N/A

Programmer must
explicitly ensure
semantics by polling
to verify completion

Message Passing Operations

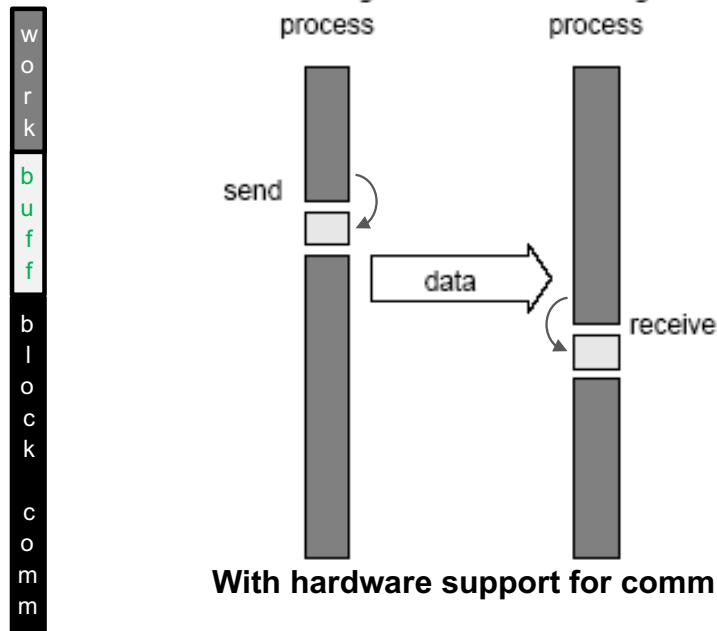
Space of Communication Protocols: **A** **B** **C**



A. Buffered Blocking Operations

(eager)

‘Send’ copies the data into designated **buffer** & **returns** only **after** completion.
Data is **buffered** at receiving end as well, which also **returns** only **after** completion.



With hardware support for comm DMA controllers, NICs, I/O processors, FPGA/ASICs, Switches and Routers, Bus Arbiters, Multicores

Buffered blocking transfer protocols

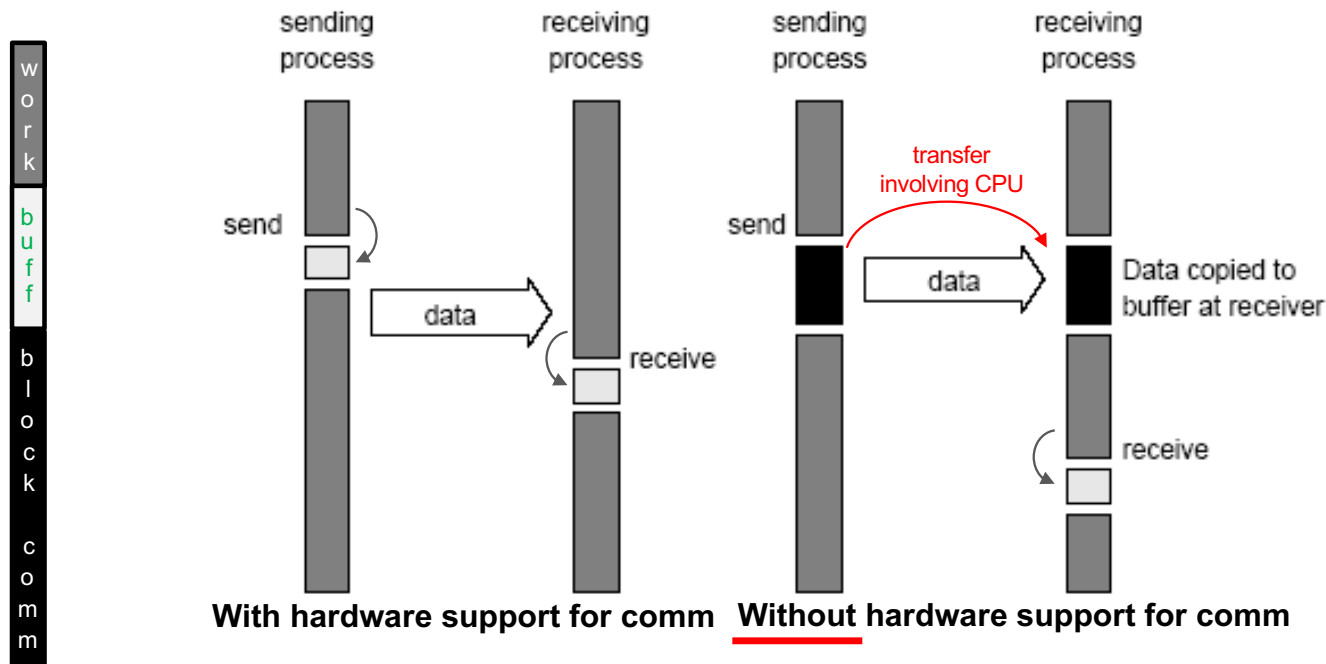
(left) Presence of communication hardware **with buffers** at **sender** and **receiver ends** (minimal **blocking**)

Buffered	Non-Blocking Operations
Sending process returns after data has been copied into communication buffer	Sending process returns after initiating DMA transfer to buffer. This operation may not be completed on return
Non-Buffered Sending process blocks until matching receive operation has been encountered	Programmer must explicitly ensure semantics by polling to verify completion
Send and Receive semantics assured by corresponding operation	

A. Buffered Blocking Operations

(eager)

‘Send’ copies the data into designated **buffer** & **returns** only **after** completion.
Data is **buffered** at receiving end as well, which also **returns** only **after** completion.



Buffered **blocking** transfer protocols

(left) Presence of communication hardware **with buffers** at **sender** and **receiver ends** (minimal **blocking**)

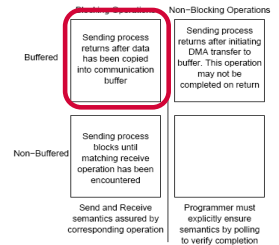
(right) No communication hardware: **CPU is involved** in **send** and **receive** to deposit data in receiver **buffer**.

In both cases, sender can only **continue** **after** data have been **buffered** (remotely or locally).

Buffered	Non-Blocking Operations
Sending process returns after data has been copied into communication buffer.	Sending process returns after initiating DMA transfer to buffer. This operation may not be completed on return.
Non-Buffered Sending process blocks until matching receive operation has been encountered. Send and Receive semantics assured by corresponding operation.	Programmer must explicitly ensure semantics by polling to verify completion.

A. Buffered Blocking Effects

(eager)



Impact on performance

P0

```
for (i = 0; i < 1000; i++){  
    produce_data(&a);  
    send(&a, 1, 1);  
}
```

P1

```
for (i = 0; i < 1000; i++){  
    receive(&a, 1, 0);  
    consume_data(&a);  
}
```

What if consumer P1 was much **slower** than producer P0 (or vice versa)?
⇒ **Idling** on sender side (conversely, receiver side).

P0

```
receive(&a, 1, 1);  
send(&b, 1, 1);
```

P1

```
receive(&a, 1, 0);  
send(&b, 1, 0);
```

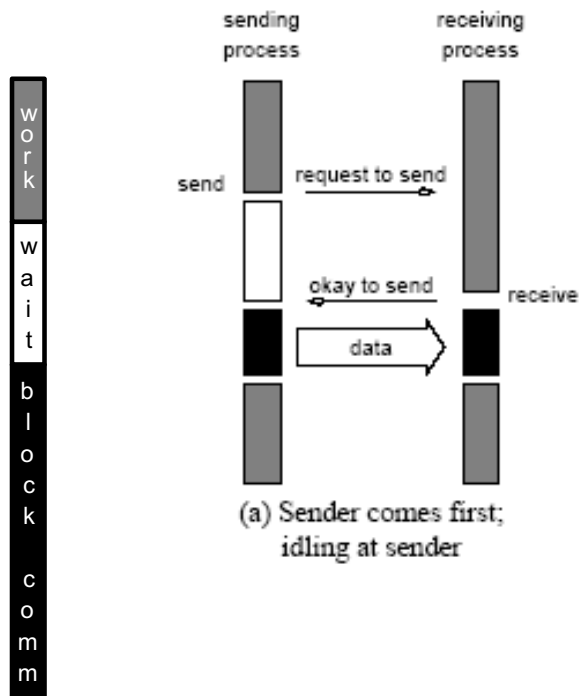
Deadlocks are still possible with **buffering** since receive operations are **blocking**.

B. Non-Buffered Blocking Operations

(rendezvous)

'Send' waits for the matching **'receive'** to be posted, and/or vice versa. **Timing** is key.

Idle scenarios due to *late* sender or *late* receiver:



Blocking Operations		Non-Blocking Operations	
Buffered	Sending process returns after data has been copied into communication buffer.	Non-Blocking Operations	Sending process returns after initiating DMA transfer to buffer. This operation may not be completed on return.
Non-Buffered	Sending process blocks until matching receive operation has been encountered.		Programmer must explicitly ensure semantics by polling to verify completion.

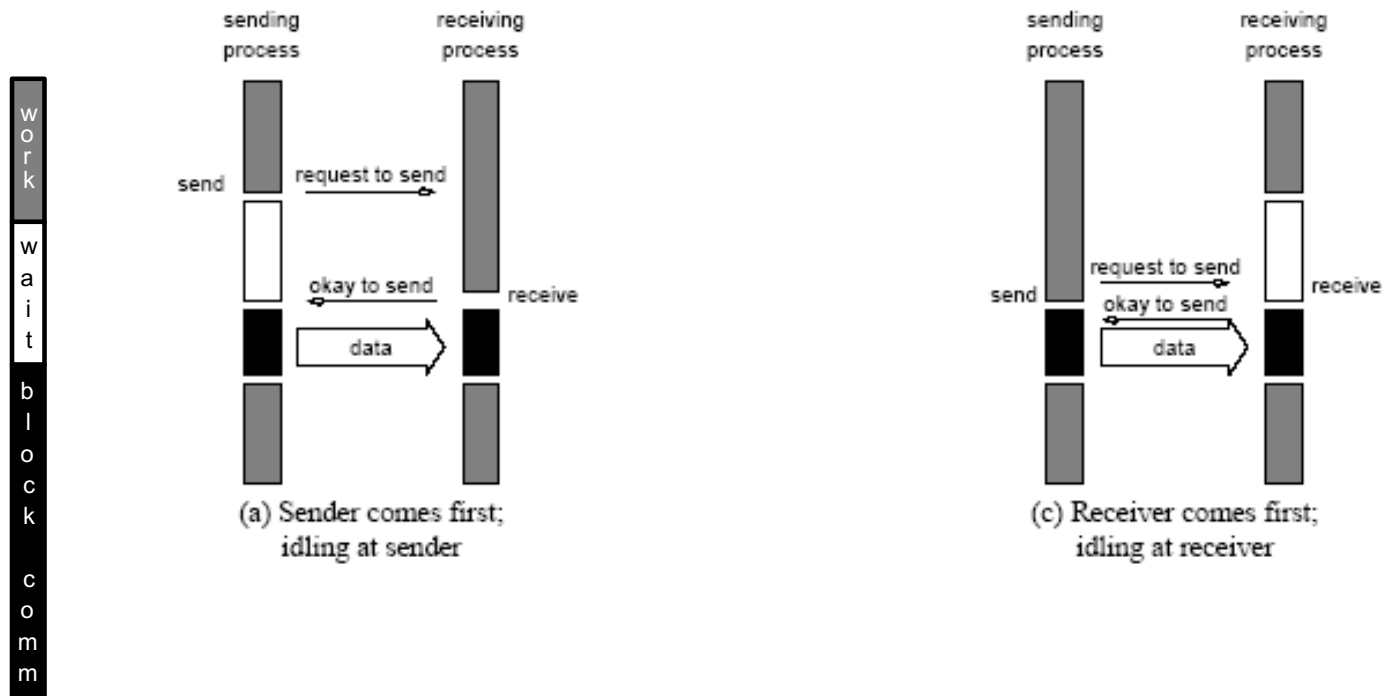
B. Non-Buffered Blocking Operations

(rendezvous)

‘Send’ **waits** for the matching ‘receive’ to be posted, and/or vice versa. **Timing** is key.

Idle scenarios due to *late* sender or *late* receiver:

Idling is in both cases a major issues with **non-buffered blocking communication**.



Blocking Operations		Non-Blocking Operations	
Buffered	Sending process returns after data has been copied into communication buffer.	Non-Blocking	Sending process returns after initiating DMA transfer to buffer. This operation may not be completed on return.
Non-Buffered	Sending process blocks until matching receive operation has been encountered.		Programmer must explicitly ensure semantics by polling to verify completion.

B. Non-Buffered Blocking Operations

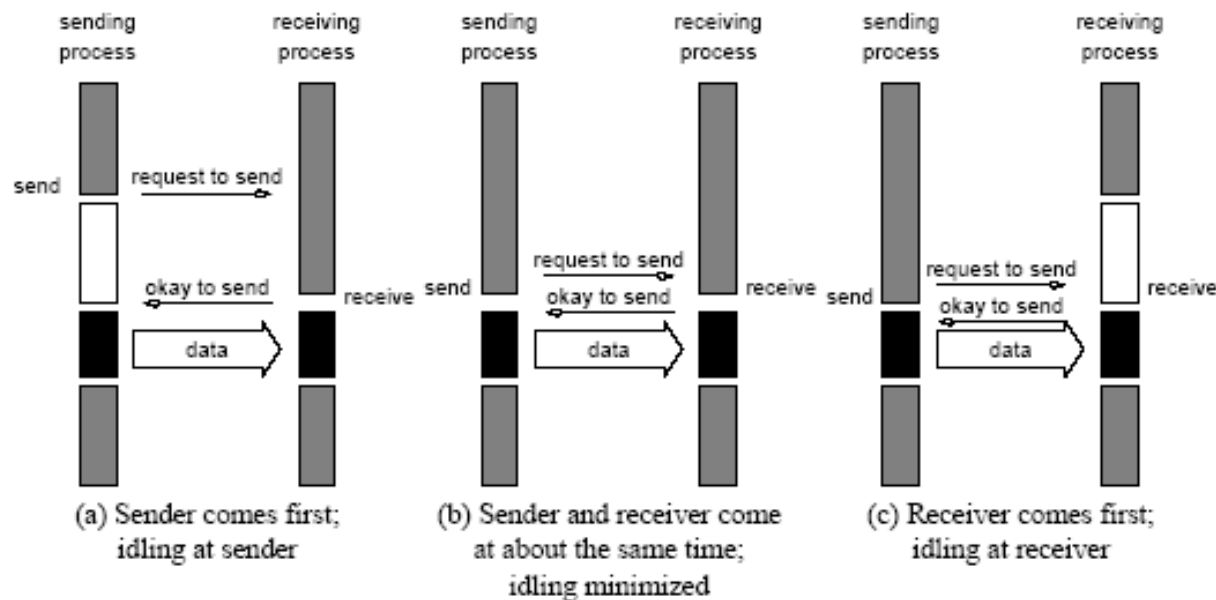
(rendezvous)

‘Send’ **waits** for the matching ‘receive’ to be posted, and/or vice versa. **Timing** is key.

Ideal scenarios minimize idling by sender and receiver being right on time.

Deadlocks are major issues with **non-buffered blocking communication**.

work
wait
block
comm



Deadlock potential

P0

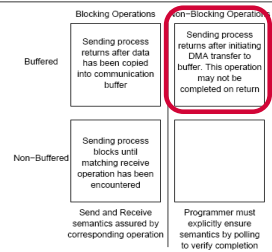
```
send(&a, 1, 1)
receive(&b, 1, 1)
```

P1

```
send(&a, 1, 0)
receive(&b, 1, 0)
```

Blocking Operations		Non-Blocking Operations	
Buffered	Sending process returns after data has been copied into communication buffer.	Non-Blocking	Sending process returns after initiating DMA transfer to buffer. This operation may not be completed on return.
Non-Buffered	Sending process blocks until matching receive operation has been encountered.		Programmer must explicitly ensure semantics by polling to verify completion.

C. Buffered Non-Blocking Operations (1/2)



Programmer must ensure semantics of ‘**send**’ and ‘**receive**’.

Non-blocking operations are generally accompanied by a **status-check** operation.

++ **Performance:** Primitives can **overlap** communication overheads with useful computations
→ addresses one (the biggest?) issue of parallel programming: *communication takes (more) time*

⇒ There is a recent trend to breaking away from loosely synchronous (BSP) towards (entirely) **asynchronous parallelism**, using **tasking** (data+work) in all its possible incarnations.

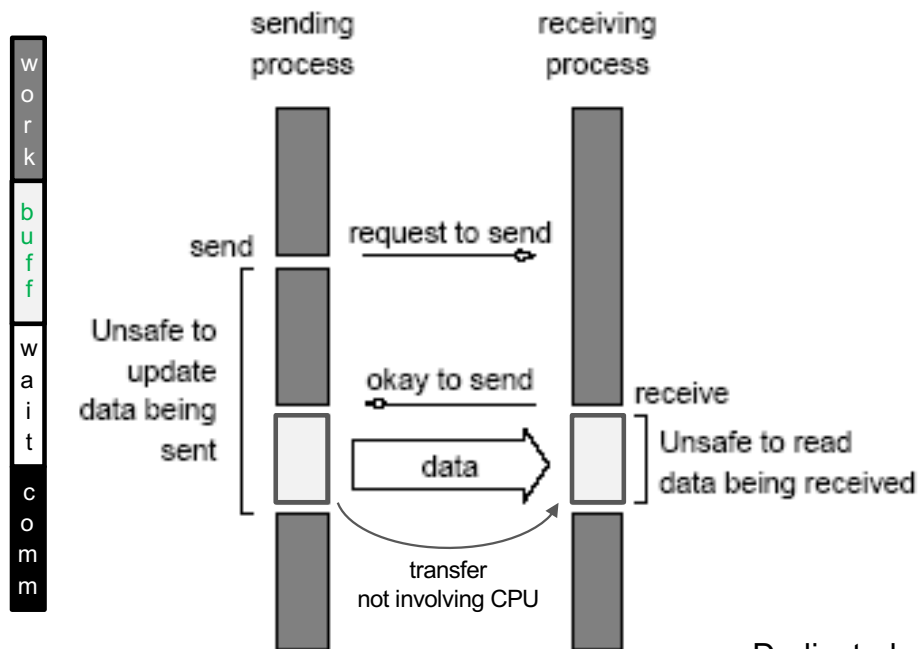
- - **Correctness:** **Non-blocking** protocols **return** from the ‘**send**’ or ‘**receive**’ operation **before it is semantically safe** to do so.



⇒ Will examine correctness more closely in Spring HPC lecture

C. Buffered Non-Blocking Operations (2/2)

The send operation is **non-blocking**



With hardware support for comm

‘**Send**’ initiates transfer request to **buffer** & **returns**.
Upon receiving OK from receiver, data transfer occurs
but does **not** **block** **either** sender/receiver.

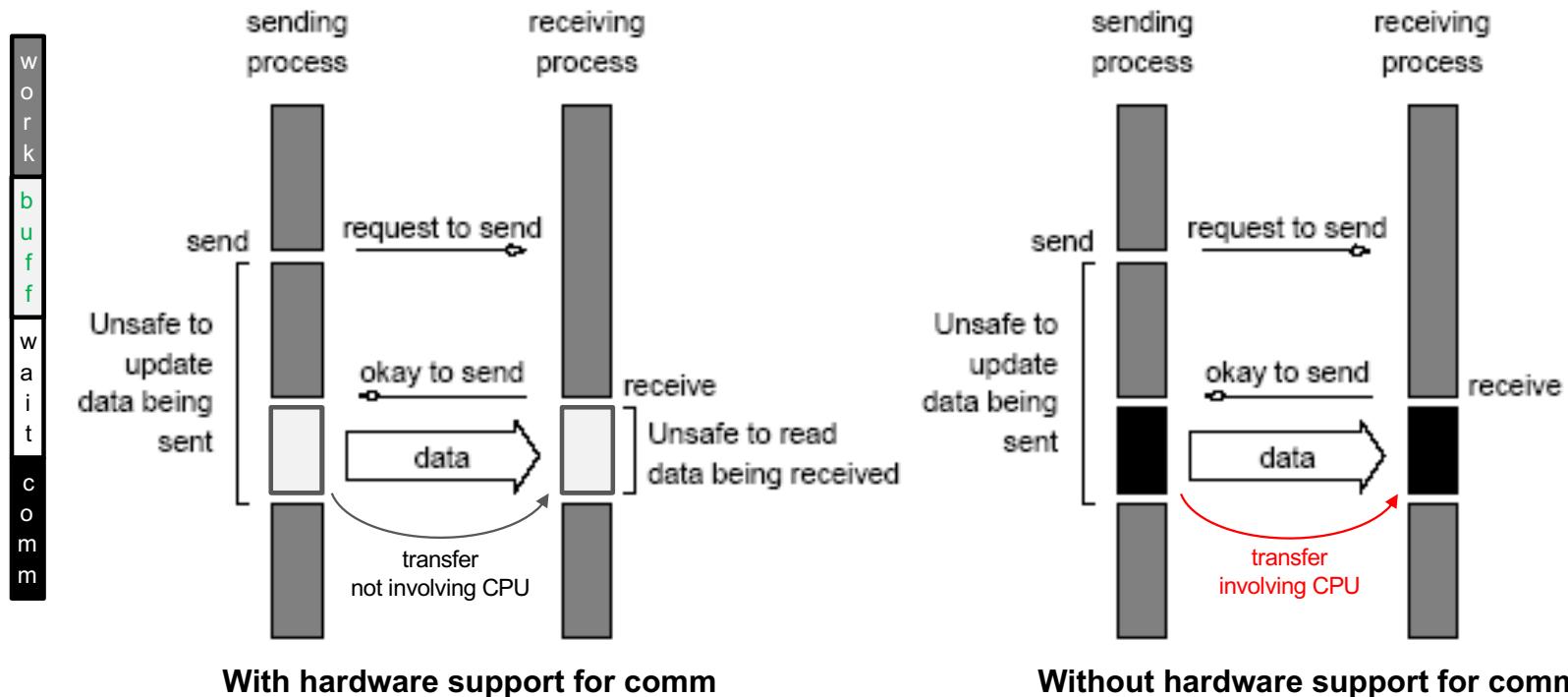
Dedicated communication channels and (possibly) synchronization mechanisms: e.g., DMA controllers, NICs, I/O processors, FPGA/ASICs, Switches and Routers, Bus Arbiters, Multicores

Blocking Operations	Blocking Operations
Buffered	Non-Blocking Operations
Sending process returns after data has been copied into communication buffer	Sending process returns after initiating DMA transfer to buffer. This operation may not be completed on return
Non-Buffered	
Sending process blocks until matching receive operation has been encountered	
Send and Receive semantics assured by corresponding operation	Programmer must explicitly ensure semantics by polling to verify completion

C. Buffered Non-Blocking Operations (2/2)

Buffered	Blocking Operations Sending process returns after data has been copied into communication buffer	Non-Blocking Operations Sending process returns after initiating DMA transfer to buffer. This operation may not be completed on return
	Non-Buffered Sending process blocks until matching receive operation has been encountered	Programmer must explicitly ensure semantics by polling to verify completion
Send and Receive semantics assured by corresponding operation		

The send operation is **non-blocking**



‘Send’ initiates transfer request to **buffer** & **returns**.
Upon receiving OK from receiver, data transfer occurs
but does **not block** either sender/receiver.

‘Send’ manages the **buffer**, initiates transfer request & **returns**.
Upon receiving OK from receiver, data transfer occurs
but **temporarily blocks** sender and receiver during transfer.

Message Passing Programming: Libraries and Standard

PVM: Parallel Virtual Machine (1991-2011)

- Typically, heterogeneous network of computers (LAN)
- Dynamic task creation; daemon-based runtime
- Historically important; largely superseded by MPI
- Docs: <http://www.csm.ornl.gov/pvm/>, <http://www.netlib.org/pvm3/book/node1.html>



MPI: The Message Passing Interface (1994-)

- Standardized communication API for parallel and/or distributed computing (C/C++/Fortran), 5.0 2025
- MPI standard defines both the *syntax* and *semantics* of core library routines for communication, and covers
 - point-to-point, collectives, one-sided, and extended collectives
 - Groups & communicators (contexts)
 - Process topologies
 - Environment & process creation and management
 - external interfaces & I/O,
 - Miscellaneous: error handling, timing (MPI_Wtime), info objects, version queries
 - Interfaces for performance tools, debuggers, profilers (MPI_T, PMPI, etc.) for analysis & debugging
- Docs: MPI Forum: www.mpi-forum.org, Tutorial: mpitutorial.org, Implementation: www.open-mpi.org



MPI Library: Start and End

MPI_Init is called prior to any other calls to the MPI library, to initialize the MPI environment.

MPI_Finalize is called at the end of the parallel and distributed computation. Performs various software clean-up operations to properly terminate the MPI environment.

⚠ No code should be executed after MPI_Finalize and before exit (return 0) from program.

The prototypes of these two functions are:

```
int MPI_Init(int *argc, char ***argv)
int MPI_Finalize()
```

Successful completion returns `MPI_SUCCESS`.

Unsuccessful completion returns `MPI_ERROR`.

Minimal Set of **MPI** Routines

It is possible to write fully-functional Message Passing programs by using only **six MPI routines**. All MPI routines, data-types, and constants are prefixed by “**MPI_**”.

MPI_Init

Initializes MPI

MPI_Comm_size

Determines the **number** of processes

MPI_Comm_rank

Determines the **ID** of calling process

MPI_Send

Sends a message

MPI_Recv

Receives a message

MPI_Finalize

Terminates MPI

MPI Groups and Communicators

Definitions

A **group** is an **ordered set** of k processes with MPI ranks ranging between $\{0..k-1\}$.

A **process group** is represented in memory by an **MPI handle**, of type **MPI_Group**. Groups can be created, duplicated, or destroyed dynamically during program execution.

A **communicator** defines a distinct communication **context/domain** (a **group** of processes that can safely communicate with each other without interference from other **communicators**).

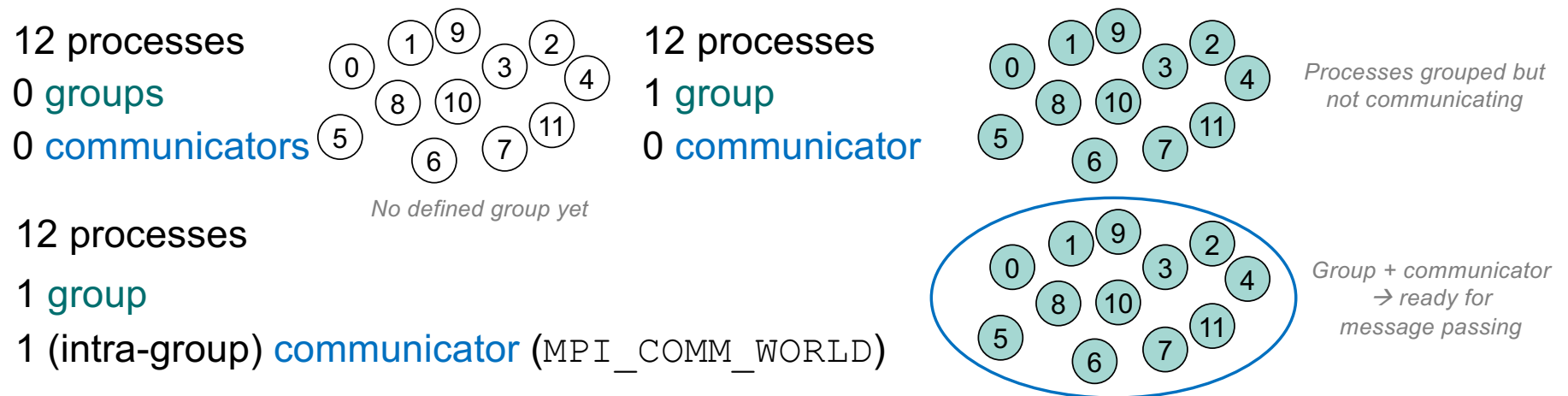
Similar to NUMA domains in shared-memory (but unlike them, MPI communicators can overlap).

All communication in MPI occurs **within a communicator**, a variable of type **MPI_Comm** that must be passed to every MPI communication call/routine.

 **Note:** An MPI process can belong to many different (possibly overlapping) communication domains (communicators). Unlike NUMA domains in shared-memory systems.

Key points: **Groups** define “**who**” can participate in message passing;
Communicators define “**where**” and “**how**” communication occurs.

MPI Groups and Communicators Visualization



MPI defines a **default communicator** called **MPI_COMM_WORLD** (of type **MPI_Comm**) which includes **all MPI processes**; by default, all communications using this communicator are **intra-group**.

 Note: In FDS-HPC we use only **intra-group communicators** (the communicator contains processes from only one group).

 A **communicator** also defines **which** processes may communicate within that group, not only **where** and **how** communication occurs.

MPI Communicators

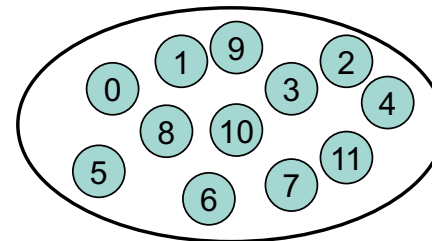
Querying Information

The `MPI_Comm_size` and `MPI_Comm_rank` are API functions used to determine the **number** (size) of processes in a communicator and the **ID** (rank) of the calling process, respectively.

```
// Example: MPI_Comm comm = MPI_COMM_WORLD;  
int MPI_Comm_size(MPI_Comm comm, int *size)  
int MPI_Comm_rank(MPI_Comm comm, int *rank)
```

Both functions return `MPI_SUCCESS` on success and are collective ops over the communicator.

The **rank** (ID) of a process is an integer that ranges from zero up to the communicator **size** minus one and uniquely identifies each process *within a communicator* (not globally).



`size=12, 0<=rank<=11`

Execution of **MPI** Programs

- From a terminal window (command line):
 - Compile source program*
`mpicc <sourcefile.c/c++/f90> -o <codefile>`
 - Execute program
`mpirun -n <nr_processes> ./<codefile>`

 *Requires a working installation of the MPI library on system
`e.g., brew install open-mpi`

Hello World with **MPI**: Version 1

```
#include "stdio.h"
#include "mpi.h"

int main(int argc, char *argv[])
{
    int npes, myrank;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &npes);
    MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
    printf("From process %d out of %d, Hello World!\n",
           myrank, npes);
    MPI_Finalize();
    return 0;
}
```



From process 2 out of 4,
Hello World!
From process 0 out of 4,
Hello World!
From process 3 out of 4,
Hello World!
From process 1 out of 4,
Hello World!

Q: How does this code execute among processes?

A: Every process runs the same program, has its own rank (ID), executes the same code independently, and uses its rank to identify itself.

Hello World with MPI: Version 2

```
#define MASTER 0
#include "stdio.h"
#include "mpi.h"

int main (int argc, char *argv[])
{
    int    numranks, rankid, len;
    char hostname[MPI_MAX_PROCESSOR_NAME];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numranks);
    MPI_Comm_rank(MPI_COMM_WORLD, &rankid);
    MPI_Get_processor_name(hostname, &len); // len = actual length of name
    printf ("Hello from rank %d on %s\n", rankid, hostname);
    if (rankid == MASTER)
        printf("MASTER: Number of MPI ranks is: %d\n", numranks);
    MPI_Finalize();
    return 0;
}
```

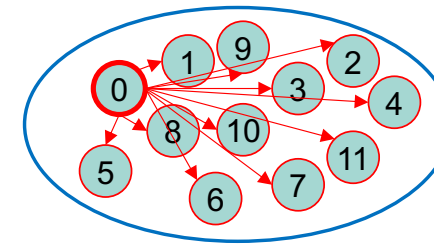
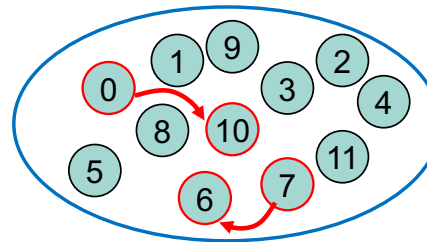
Q: How does this code execute among processes?

A: Ranks can perform different actions.
Processes may run on different nodes.



Hello from rank 1 on 25-1a-0232.client.unibas.ch
Hello from rank 3 on 25-1a-0232.client.unibas.ch
Hello from rank 0 on 25-1a-0232.client.unibas.ch
MASTER: Number of MPI ranks is: 4
Hello from rank 2 on 25-1a-0232.client.unibas.ch

MPI Communications

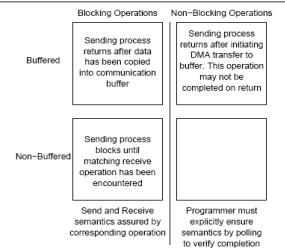


	Blocking Operations	Non-Blocking Operations
Buffered	Sending process returns after data has been copied into communication buffer	Sending process returns after initiating DMA transfer to buffer. This operation may not be completed on return
Non-Buffered	Sending process blocks until matching receive operation has been encountered	Programmer must explicitly ensure semantics by polling to verify completion
	Send and Receive semantics assured by corresponding operation	

Types	Point-to-point	Collective
Blocking	default	default
Non-blocking	increasingly common persistent	<i>more in HPC lecture Spring</i>

These are incarnations of Message Passing *principles* in the [MPI standard](#) and available in [MPI implementations](#) (Open MPI, MPICH, etc.).

MPI Communications: Point-to-point and Collectives



Communication	Point-to-point	Collective
Modes	Synchronous Buffered Ready Standard	Synchronization Data movement Global computation
Types (previous table)	Blocking (until completing locally) Non-blocking + persistent	Blocking (until completing locally) Non-blocking since MPI 3.0
Participants	One-to-one (point-to-point)	Barrier One-to-many Many-to-one Some-to-some All-to-all
Characteristics	Involve coordinated communication between two processes (sender and receiver) Uses message tags	Involve coordinated communication within a group of processes No message tags

MPI Point-to-Point: Communication Modes

Choice of mode depends on **communication overhead** = transfer + synchronization

Communication mode	Blocking (until completing locally) routines	Non-blocking & persistent _init routines
Synchronous *	MPI_ S send (RDV)	MPI_ I ssend, MPI_ S send_init
Buffered	MPI_ B send (Eager)	MPI_ I bsend, MPI_ B send_init
Ready	MPI_ R send [imposes late Sender]	MPI_ I rsend, MPI_ R send_init
Standard	MPI_Send implemented as {MPI_ S send** or MPI_ B send***}	MPI_Isend (most common) implemented as {MPI_ I ssend** or MPI_ I bsend***}, MPI_Send_init
	MPI_Recv (most common)	MPI_ I recv, MPI_ Y recv_init
	MPI_Sendrecv implemented as {MPI_Send+MPI_Recv}****	MPI_ S tart, MPI_ S tartall MPI_ W ait(all any), MPI_ T est(all any)
	MPI_Sendrecv_replace *****	MPI_ C ancel, MPI_ R quest_free

* Uses the **rendezvous** protocol (sending process **blocks** locally until matching receive operation has been encountered in another process)

** *Synchronous send* for large messages (that do not fit in the communication **buffer**)

*** *Buffered send* for small messages (that fit in the communication **buffer**) and uses the **eager** protocol (sending process **blocks** locally until message is copied to send **buffer**)

**** *Sendrecv* uses different **buffers** for the send message and the receive message, respectively

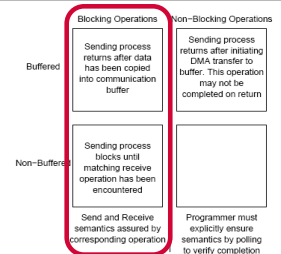
***** *Sendrecv_replace*: **buffer** used to send the message is being *overwritten* by the received message

MPI Point-to-Point: Communication Modes Pros & Cons

Mode	Advantages	Disadvantages
Synchronous	- Safest, therefore most portable - No need for extra buffer space - SEND/RECV order not critical	- Can incur substantial synchronization <i>overhead</i>
Buffered	- Decouples SEND from RECV - No synchronization overhead on SEND - Programmer can control size of buffer - SEND/RECV execution order irrelevant	- Copying to buffer incurs additional <i>system overhead</i>
Ready	- Lowest total overhead - No need for extra buffer space - SEND/RECV handshake not required as RECV always ready/first	- RECV <i>must</i> precede SEND ⚠ - Impractical to guarantee this without specific logic (e.g., Barrier)
Standard	- Good for many cases - Compromise position	- Your program may not be suitable for a generic approach, and you will have to use a different mode (sync, buffered, ready)

MPI Communications

A+B. **Blocking** Point-to-Point: Sending and Receiving



Blocking **send** and **receive** MPI calls transfer data between pairs of processes.

The calls return only after the communication buffer can be safely reused.

```
int MPI_Send(void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)
```

```
int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int src, int tag, MPI_Comm comm, MPI_Status *st)
```

MPI provides equivalent **datatypes** for all C/C++ datatypes, for portability (e.g., MPI_BYTE is a byte (8 bits), MPI_PACKED is a collection of data items that has been created by packing non-contiguous data, MPI_INT, MPI_DOUBLE).

Messages carry a **tag** (0..MPI_TAG_UB) to distinguish multiple communications.

Communication **may** (eager) / **may not** (RDV) be **buffered**, depending on implementation, which decides about buffering/not.

⇒ Important for performance and correctness when porting programs on systems with differing MPI implementations.

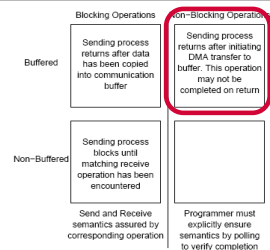
When both **send** and **receive** happen **together** (halo exchange), MPI offers combined primitives:

```
int MPI_Sendrecv(void *sndbuf, int sndcount, MPI_Datatype snddt, int dest, int sndtg, void *rcvbuf, int rcvcount, MPI_Datatype rcvdt, int src, int rcvtg, MPI_Comm comm, MPI_Status *st)
```

```
int MPI_Sendrecv_replace(void *buf, int count, MPI_Datatype datatype, int dest, int sndtag, int src, int rcvtg, MPI_Comm comm, MPI_Status *st) // The same buffer is used for both send and receive
```

MPI Communications

C. Non-blocking Point-to-Point: Sending and Receiving



Enable **overlap** of communication and computation, such that processes continue useful work while data transfer proceeds in the background.

- Allows communication and computation to proceed **concurrently** instead of BSP style: comp-comm-comp
- Enables latency hiding, not communication avoidance; helps avoid deadlocks
- Reduces system **buffering** and memory copying overhead → performance gain

```
int MPI_Isend(void *buf, int cnt, MPI_Datatype dt, int dst, int tg, MPI_Comm comm, MPI_Request *req)
int MPI_Irecv(void *buf, int cnd, MPI_Datatype dt, int src, int tg, MPI_Comm comm, MPI_Request *req)
```

- **MPI_Request** is a handle used to track each outstanding communication, through which all progress and completion are managed (`MPI_REQUEST_NULL` represents an inactive or null request).
- Both **non-blocking** operations return immediately, before the transfer finishes.

```
MPI_Isend(...); MPI_Irecv(...); //comm
// Do useful work //comp
MPI_Test(&req, &flag, &st); //check completion
MPI_Wait(&req, &st); //block until done
```

⚠ Do not access/modify buffers involved in **non-blocking** ops until completion (**MPI_Test** or **MPI_Wait**)

MPI Communications

C. Non-blocking Point-to-Point: Sending and Receiving Properties

Order: non-overtaking, preserved

– According to the execution order of non-blocking calls that initiate the communications

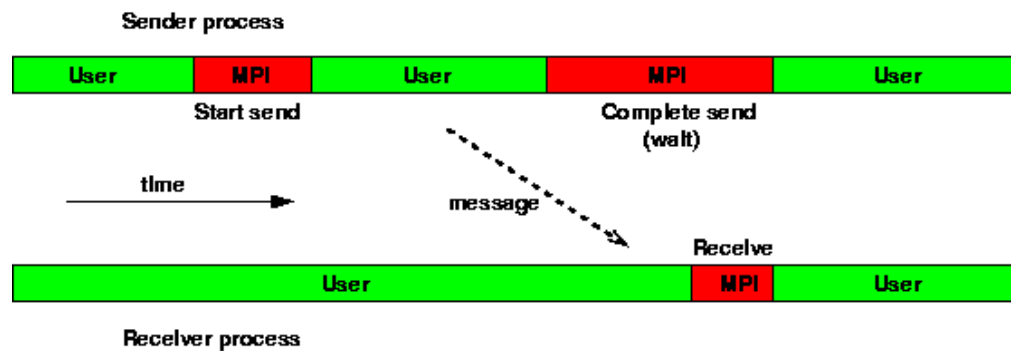
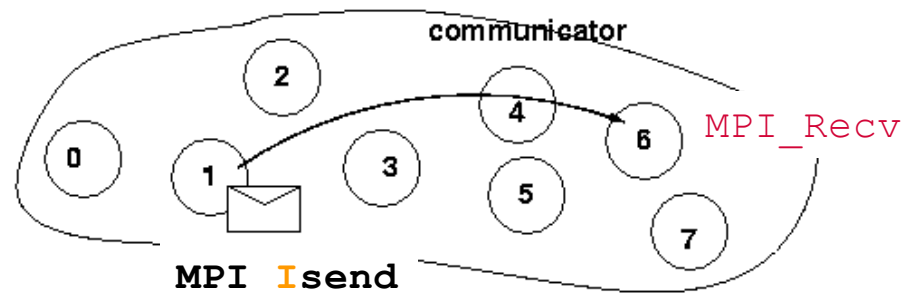
Progress: guarantees progress

- **MPI_Irecv** call completed by **MPI_Wait** will eventually return if there is a matching (non-/blocking) send.
- **MPI_Isend** call completed by **MPI_Wait** will eventually return if there is a matching (non-/blocking) receive.

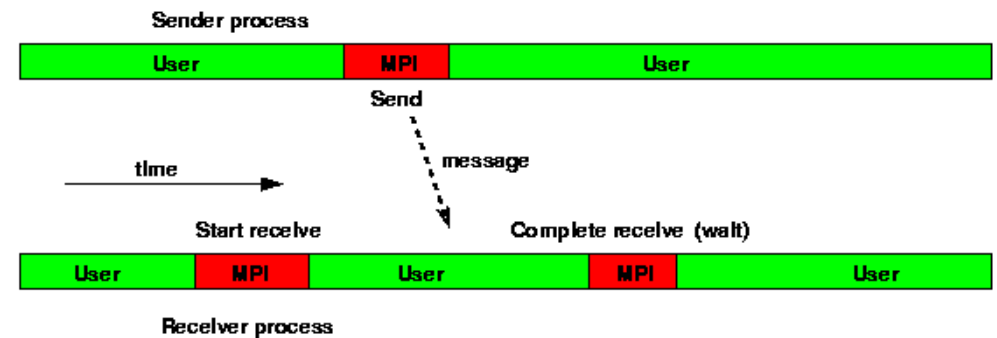
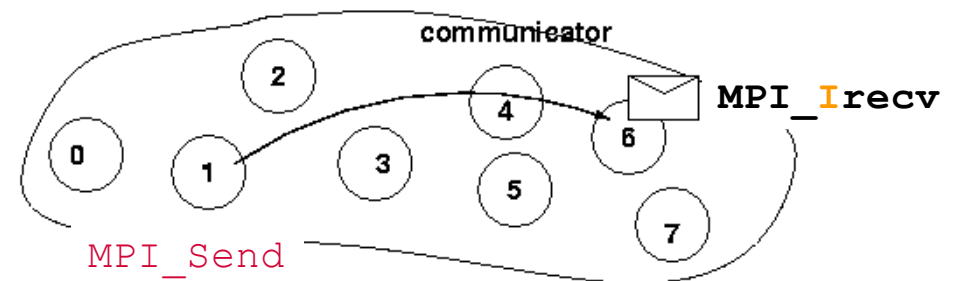
```
MPI_Comm_rank(comm, &rank);
if(rank==0) {
    MPI_Isend(A, 1, MPI_DOUBLE, 1, 99, comm, &req1);
    MPI_Isend(B, 1, MPI_DOUBLE, 1, 99, comm, &req2);
}
else if(rank==1) {
    MPI_Irecv(A, 1, MPI_DOUBLE, 0, MPI_ANY_TAG, comm, &req1);
    MPI_Irecv(B, 1, MPI_DOUBLE, 0, 99, comm, &req2);
}
MPI_Wait(&req1, &stat1);
MPI_Wait(&req2, &stat2);
```

MPI Communications

C. Non & blocking Point-to-Point: Sending and Receiving Combinations



Non-blocking sender “waits” for late blocking receiver scenario

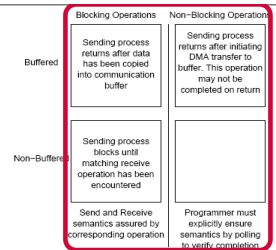


Non-blocking receiver “waits” for late blocking sender scenario

! Non-blocking communication overlaps with computation and *avoids synchronization stalls* (as with blocking cases).

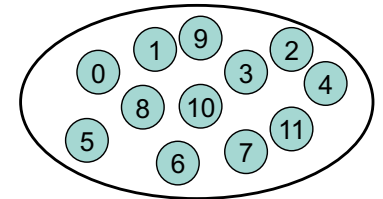
MPI Communications

A+B+C: Collectives



MPI provides an extensive set routines for common **collective communication operations**, actions involving all processes in a **communicator** (can be the same as for point-to-point operations)

- Each operation is defined over a **group** of processes (associated with a **communicator**)
- **All processes** in a **group must participate** in the same collective call with matching arguments
- MPI guarantees that collective messages **will not be confused** with point-to-point messages on the same communicator
- If only a subset of processes need participate, create a **sub-communicator** (**subgroup** of `MPI_COMM_WORLD`).



Operation Modes

- Synchronization: `MPI_Barrier`
- Data movement: `MPI_Bcast`, `MPI_Scatter`, `MPI_Gather`, `MPI_Allgather`, `MPI_Alltoall`
- Collective computation: `MPI_Reduce`, `MPI_Allreduce`, `MPI_Scan`
- Up to MPI 2.0, collective operations were **blocking** to ensure correctness. Completion implied the buffer was **safe to access**, though other processes' states may differ.
- From **MPI 3.0 onward**, **non-blocking collectives** (`MPI_Ibcast`, `MPI_Ireduce`) **overlap** comm w/ comp. May or may not include **implicit synchronization** among participating processes. (more in Spring HPC)

Collectives **simplify communication** by replacing multiple point-to-point calls with 1 coordinated operation.

MPI Communications

A+B+C: Collectives: Communication Modes

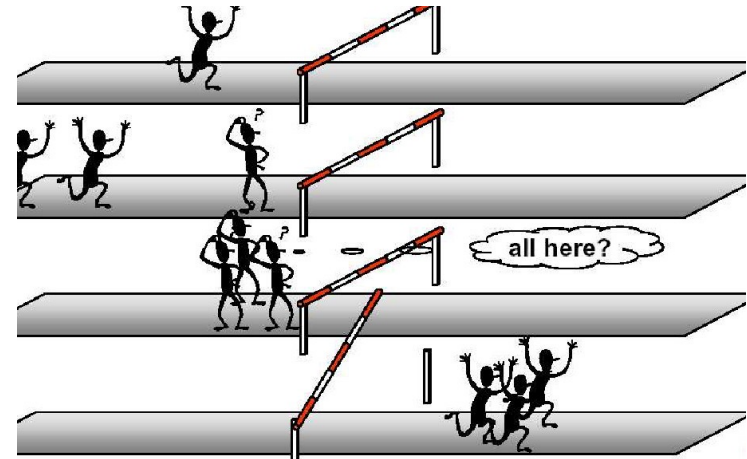
Modes	Participants	Routines (blocking)	Routines (non-blocking)	Characteristics
Synchronization	barrier	MPI_Barrier	MPI_ barrier	• Barrier synchronization across ranks in communicator
Data movement	one-to-many	MPI_Bcast MPI_ Scatter, MPI_ Scatterv	MPI_ bcast MPI_ scatter, MPI_ scatterv	• Broadcast or scatter data from one to all members in communicator
	many-to-one	MPI_ Gather, MPI_ Gatherv	MPI_ gather, MPI_ gatherv	• Gather data spread across processes into one array (on rank 0)
	all-to-all	MPI_ Allgather, MPI_ Allgatherv MPI_ Alltoall, MPI_ Alltoallv, MPI_ Alltoallw	MPI_ allgather, MPI_ allgatherv MPI_ alltoall, MPI_ alltoallv, MPI_ alltoallw	• Gather data or all-to-all data exchange among all ranks supports varied buffer sizes
	many-to-one + one-to-many	MPI_ Reduce_scatter MPI_ Reduce_scatter_block	MPI_ reduce_scatter MPI_ reduce_scatter_block*	• Perform global reduction (sum, min, max) • Reduction followed by broadcast to all ranks • Combine reduction and scatter of results (*equally)
Global computation	some-to-some	MPI_ Scan, MPI_ Exscan	MPI_ scan, MPI_ exscan	• Computes an inclusive/exclusive scan (partial reduction) across members of a communicator

! **Non-blocking** collectives are initiated with an **| prefix** and return immediately with `MPI_Request` handle. Completion managed similarly to non-blocking point-to-point with `MPI_Test` and `MPI_Wait`.

MPI Communications

A+B+C: Collectives: Synchronization > Barrier

```
int MPI_Barrier(MPI_Comm comm)
```



Barrier is *collective operation*: all processes in the communicator must call it, and **blocks** each calling process until **all group members** have called it.

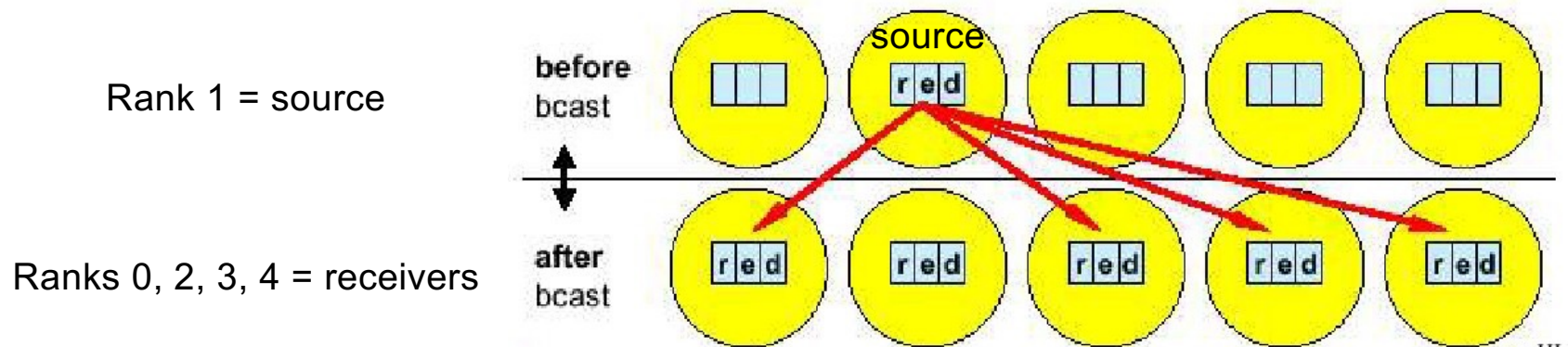
⚠ Decreases performance (but increases correctness).

Use sparingly, only when strict synchronization is required.

```
...  
MPI_Barrier(MPI_COMM_WORLD); // synchronization point  
...
```

MPI Communications

A+B+C: Collectives: Data Movement > One-to-Many > **Broadcast**



```
int MPI_Bcast(void *buf, int count,  
             MPI_Datatype datatype, int source, MPI_Comm comm);
```

Broadcasts a message from a process, e.g., 1, to all processes in group, **including itself**.

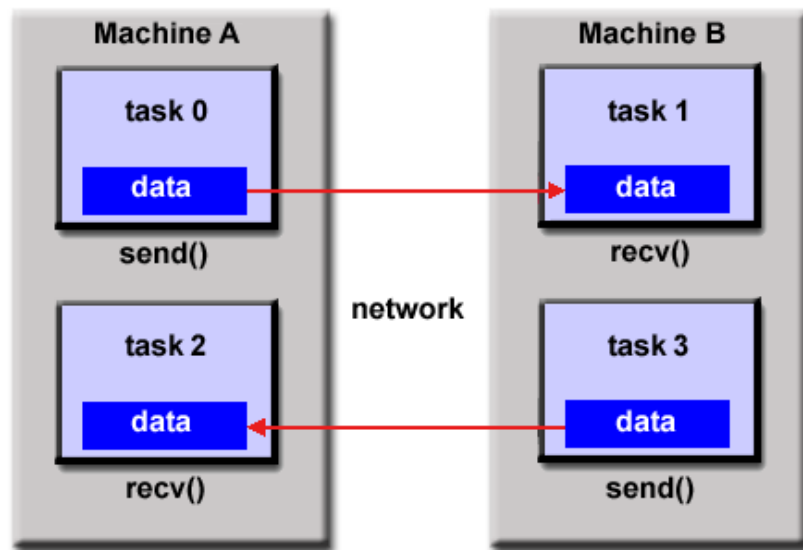
⚠ All processes must use the same `comm`, `source` values.

! `Count` and `datatype` must match `source` across processes.

- MPI checks this consistency pairwise at runtime
- When using MPI derived datatypes, the actual memory layouts may differ b/w processes, and actual data size depends on the datatype definition, even if `count` and `datatype` look the same

```
MPI_Bcast(array, 100, MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

What Have We Learned So Far?



Parallel programming model 4:
Distributed-Memory | Message passing

Introduction to Parallel Programming

- a Shared-Memory Programming: OpenMP Intro
- b Distributed-Memory Programming: MPI Intro

Summary

- BSP (Bulk Synchronous Parallel) loosely synchronous mode
- **MPI** has many pros but also some cons
- Message Passing Communication Protocols
 - Buffered, non-buffered
 - Blocking, non-blocking
- **MPI** communication primitives
 - Point-to-point, persistent, collective communication
 - (buffered, non-buffered) × (blocking and non-blocking)
- Idling and hardware support for communication affect **efficiency**
- Deadlocks affect **correctness**

What Have We Learned In This Sub-Chapter 5.3?

End of 5.3 ⇒ “OMP_NOWAIT + MPI_Finalize()”

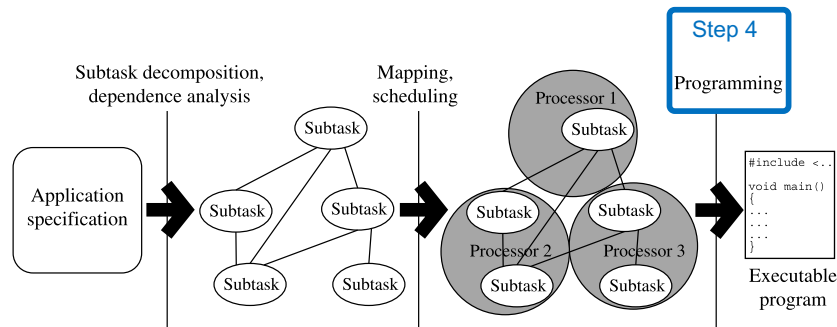


Figure 1.1. Parallel programming—process of parallelization.

Introduction to Parallel Programming

- a Shared-Memory Programming: OpenMP Intro
- b Distributed-Memory Programming: MPI Intro

Summary

- Now you can **Compute** in parallel!
 - **Communication** is about who needs what and when (for computing as decided above)
 - **Locality**: is data close to processing (in memory)
 - **Load imbalance** is everyone progressing uniformly
- **Parallel programming** by extending languages: low-level approach, with its pros/cons
- Fork-join parallelism and **OpenMP**: **D**irectives, **R**untime, **E**nvironment
 - **D**: Parallel regions + Work sharing, **R**untime library functions, **E**nvironment variables
- **BSP** (Bulk Synchronous Parallel) is parallel computing in a loosely synchronous mode
- **MPI communications**: (buffered, non-buffered) × (blocking and non-blocking)
 - Point-to-point, persistent, and collective communication

Overview of Chapter 5.3: Parallel Programming

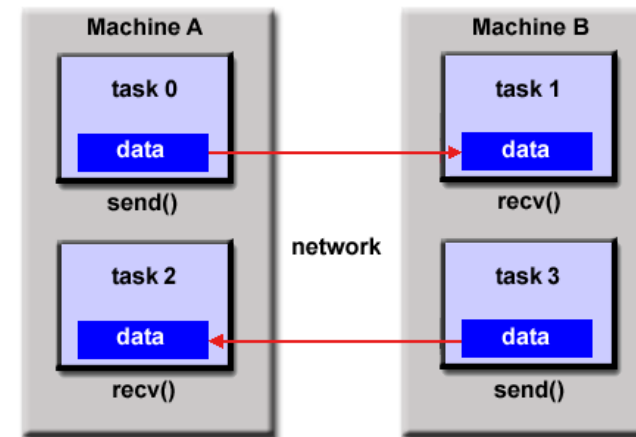
Introduction to Parallel Programming

a Shared-Memory Programming: OpenMP Introduction

b Distributed-Memory Programming: MPI Introduction – MPI calls examples

Parallel programming model 4:
Distributed-Memory | Message passing

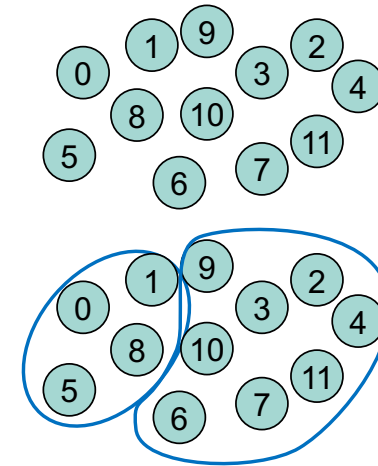
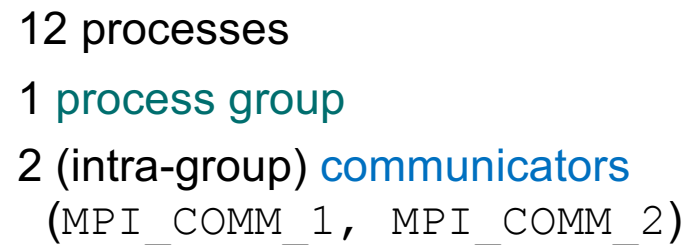
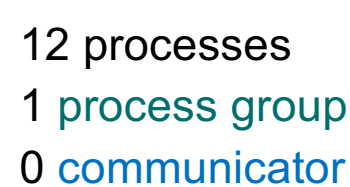
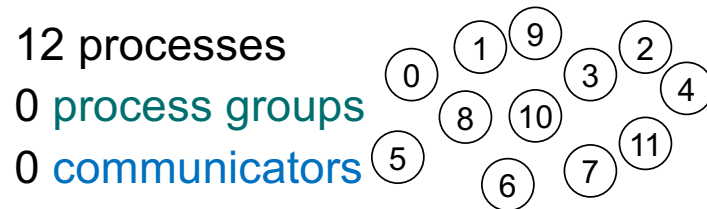
Additional Material starts here



Disclaimer: This lecture introduced **concepts** of distributed-memory programming.

⚠ Practical examples are included in these additional slides to complement the concepts in a self-contained manner, and to be used for independent self-study, not for in-class teaching.

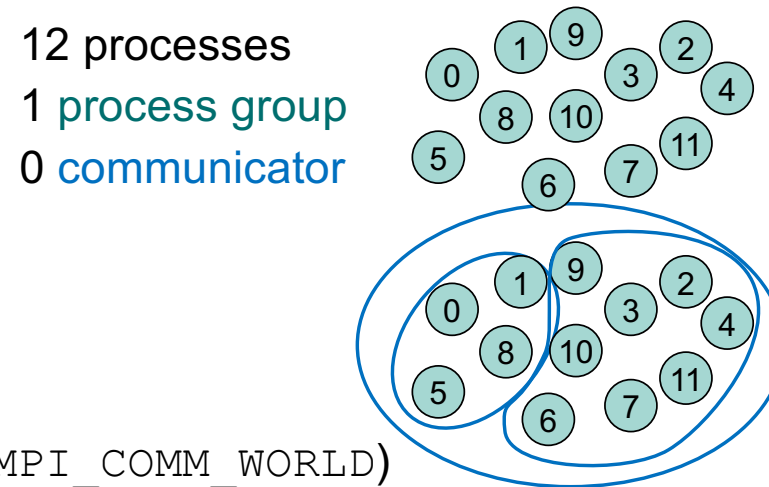
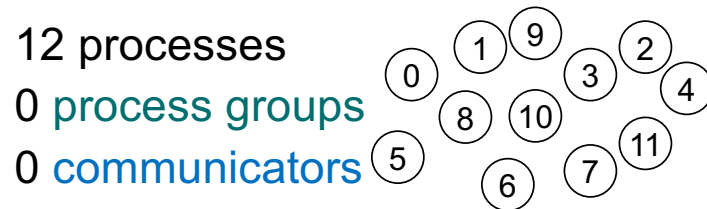
MPI Sub-Communicators



MPI defines a **default communicator** called **MPI_COMM_WORLD** (of type **MPI_Comm**) which includes all MPI processes.

 Note: In this course we use only **intra-group communicators** (communicator contains processes from only one group).

MPI Sub-Communicators



MPI defines a **default communicator** called **MPI_COMM_WORLD** (of type **MPI_Comm**) which includes all MPI processes.

 Note: In this course we use only **intra-group communicators** (communicator contains processes from only one group).

MPI Datatypes



MPI Datatype	C Datatype
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	
MPI_PACKED	

Further datatypes exist, see
Annex on MPI forum
(<https://www.mpi-forum.org>)

MPI Example: Ping



```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int numtasks, rank, dest, source, rc, tag = 1;
    char outmsg = 'A', inmsg = '?';

    MPI_Status Stat;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (numtasks < 2) {
        if (rank == 0) fprintf(stderr, "Need at least 2 processes.\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
    if (rank == 0) { // MASTER
        if (numtasks > 2)
            printf("Numtasks=%d. Only ranks 0 and 1 participate.\n", numtasks);
        dest = 1; source = 1;
        rc = MPI_Send(&outmsg, 1, MPI_CHAR, dest, tag, MPI_COMM_WORLD);
        rc = MPI_Recv(&inmsg, 1, MPI_CHAR, source, tag, MPI_COMM_WORLD, &Stat);
        printf("Rank 0 received '%c' from rank %d (tag %d)\n",
            inmsg, Stat.MPI_SOURCE, Stat.MPI_TAG);
    } else if (rank == 1) { // WORKER
        dest = 0; source = 0;
        rc = MPI_Recv(&inmsg, 1, MPI_CHAR, source, tag, MPI_COMM_WORLD, &Stat);
        rc = MPI_Send(&outmsg, 1, MPI_CHAR, dest, tag, MPI_COMM_WORLD);
        printf("Rank 1 received '%c' from rank %d (tag %d)\n",
            inmsg, Stat.MPI_SOURCE, Stat.MPI_TAG);
    }
    // Ranks >= 2 do nothing but still must finalize.
    MPI_Finalize();
    return 0;
}
```

Status



- For a receive operation, information may be requested via status record `MPI_Status`
- `MPI_SOURCE`, `MPI_TAG`
- `MPI_GetCount`: Request number of information elements received and get this in `&Stat`.

```
rc = MPI_Get_count(&Stat, MPI_CHAR, &count);  
  
printf("Task %d: Received %d char(s) from task %d with tag %d \n",  
       rank, count, Stat.MPI_SOURCE, Stat.MPI_TAG);
```

Deadlock Potential (1)



```
int a[10], b[10], myrank;

...
MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
if (myrank == 0) {
    MPI_Send(a, 10, MPI_INT, 1, 1, MPI_COMM_WORLD);
    MPI_Send(b, 10, MPI_INT, 1, 2, MPI_COMM_WORLD);
}
else if (myrank == 1) {
    MPI_Recv(b, 10, MPI_INT, 0, 2, MPI_COMM_WORLD);
    MPI_Recv(a, 10, MPI_INT, 0, 1, MPI_COMM_WORLD);
}
...
```

If `MPI_Send` is not buffered, there is a deadlock.

Can you figure out why? Hint: Semantics of `MPI_Send` and `MPI_Recv`.

Deadlock Potential (2)



The following example shows communication in a process ring.

To be distinguished from a ring of processors. To be the same, we need to map a ring of processes to a ring of processors through step 3 of parallelization: Mapping/Assignment.

```
int a[10], b[10], npes, myrank;

...
MPI_Comm_size(MPI_COMM_WORLD, &npes);
MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
MPI_Send(a, 10, MPI_INT, (myrank+1)%npes, 1, MPI_COMM_WORLD);
MPI_Recv(b, 10, MPI_INT, (myrank-1+npes)%npes, 1, MPI_COMM_WORLD);
...
```

Once again, we have a deadlock if `MPI_Send` is not buffered.

Deadlock Prevention



```
int a[10], b[10], npes, myrank;
...
MPI_Comm_size(MPI_COMM_WORLD, &npes);
MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
if (myrank%2 == 1) {
    MPI_Send(a, 10, MPI_INT, (myrank+1)%npes, 1,
             MPI_COMM_WORLD);
    MPI_Recv(b, 10, MPI_INT, (myrank-1+npes)%npes, 1,
             MPI_COMM_WORLD);
}
else {
    MPI_Recv(b, 10, MPI_INT, (myrank-1+npes)%npes, 1,
             MPI_COMM_WORLD);
    MPI_Send(a, 10, MPI_INT, (myrank+1)%npes, 1,
             MPI_COMM_WORLD);
}
...
```

We can break the circular wait to avoid deadlocks as follows:
Split into 2 groups even and odd processes which have different Send-Recv operation sequences.

Wildcard Arguments

MPI allows specification of **wildcard arguments** for both source and tag.

If source is set to `MPI_ANY_SOURCE`, then any process of the communication domain can be the source of the message.

If tag is set to `MPI_ANY_TAG`, then messages with any tag are accepted.

Example: Odd-Even Sort (1) with MPI_Sendrecv



```
...
if (myrank%2 == 0) {
    oddrank  = myrank-1;
    evenrank = myrank+1;
}
else {
    oddrank  = myrank+1;
    evenrank = myrank-1;
}

/* Set the ranks of the processors at the end of the linear */
if (oddrank == -1 || oddrank == npes)
    oddrank = MPI_PROC_NULL;
if (evenrank == -1 || evenrank == npes)
    evenrank = MPI_PROC_NULL;
...
```

Example: Odd-Even Sort (2) with MPI_Sendrecv



```
...
/* Get into the main loop of the odd-even sorting algorithm */
for (i=0; i<npes-1; i++) {
    if (i%2 == 1) /* Odd phase */
        MPI_Sendrecv(elmnts, nlocal, MPI_INT, oddrank, 1, relmnts,
            nlocal, MPI_INT, oddrank, 1, MPI_COMM_WORLD, &status);
    else /* Even phase */
        MPI_Sendrecv(elmnts, nlocal, MPI_INT, evenrank, 1, relmnts,
            nlocal, MPI_INT, evenrank, 1, MPI_COMM_WORLD, &status);

    CompareSplit(nlocal, elmnts, relmnts, wspace, myrank <
        status.MPI_SOURCE);
}
```

Example: Use of SendRecv_replace



```
...
for(count=0;count<ncpus;count++){

    // compute on current data
    curr_block = (my_rank+count)%ncpus;
    sindex = curr_block*Nx;
    for(i=0;i<Nx;i++)
        for(j=0;j<Nx;j++)
            Y[i] += A[i][sindex+j]*X[j];

    // send-recv
    if(count<ncpus-1)
        MPI_Sendrecv_replace(X, Nx, MPI_DOUBLE, left_neighbor, tag,
                              right_neighbor, tag, MPI_COMM_WORLD, &status_sr);
}
...
```



Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    int my_rank, ncpus;
    int left_neighbor, right_neighbor;
    int data_received=-1;
    int tag = 101;
    MPI_Status statSend, statRecv;
    MPI_Request reqSend, reqRecv;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &ncpus);

    left_neighbor = (my_rank-1 + ncpus)%ncpus;
    right_neighbor = (my_rank+1)%ncpus;
    // comm start
    MPI_Isend(&my_rank,1,MPI_INT,left_neighbor,tag,MPI_COMM_WORLD,&reqSend);
    MPI_Irecv(&data_received,1,MPI_INT,right_neighbor,tag,MPI_COMM_WORLD,&reqRecv);
    if (my_rank==0) printf("Maybe do something useful here\n"); // something useful
    MPI_Wait(&reqSend, &statSend); // complete send
    MPI_Wait(&reqRecv, &statRecv); // complete receive
    printf("Among %d processes, process %d received from right neighbor: %d\n",
           ncpus, my_rank, data_received);
    // clean up
    MPI_Finalize();
    return 0;
}
```

```
mpicc test-shift-mpi.c -o test-shift-mpi
mpirun -np 4 test-shift-mpi
```

Maybe do something useful here

Among 4 processes, process 3 received from right neighbor: 0

Among 4 processes, process 2 received from right neighbor: 3

Among 4 processes, process 0 received from right neighbor: 1

Among 4 processes, process 1 received from right neighbor: 2

Overview of Chapter 5.3: Parallel Programming

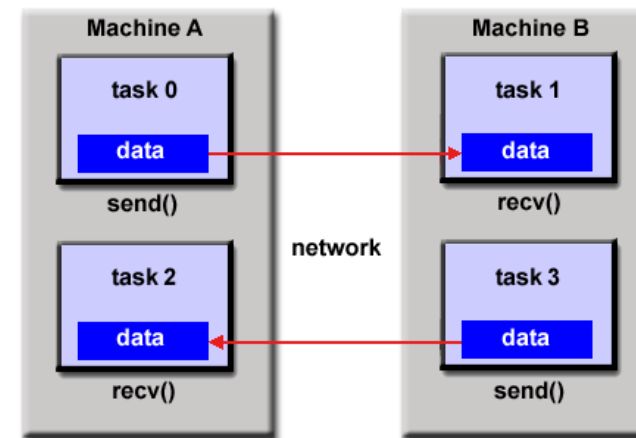
Introduction to Parallel Programming

a Shared-Memory Programming: OpenMP Introduction

b Distributed-Memory Programming: MPI Introduction – MPI calls examples

Parallel programming model 4:
Distributed-Memory | Message passing

Additional Material ends here



Disclaimer: This lecture introduced **concepts** of distributed-memory programming.

⚠ Practical examples are included in these additional slides to complement the concepts in a self-contained manner, and to be used for independent self-study, not for in-class teaching.

